endava

By Carlos Gamez

Programación Dinámica.

Objetivos

01 Definir conceptos claves de la programación dinámica.

02 Ejemplificar de forma practica la programación dinámica.

03 Aprender a identificar un problema de programación dinámica.

Chapter

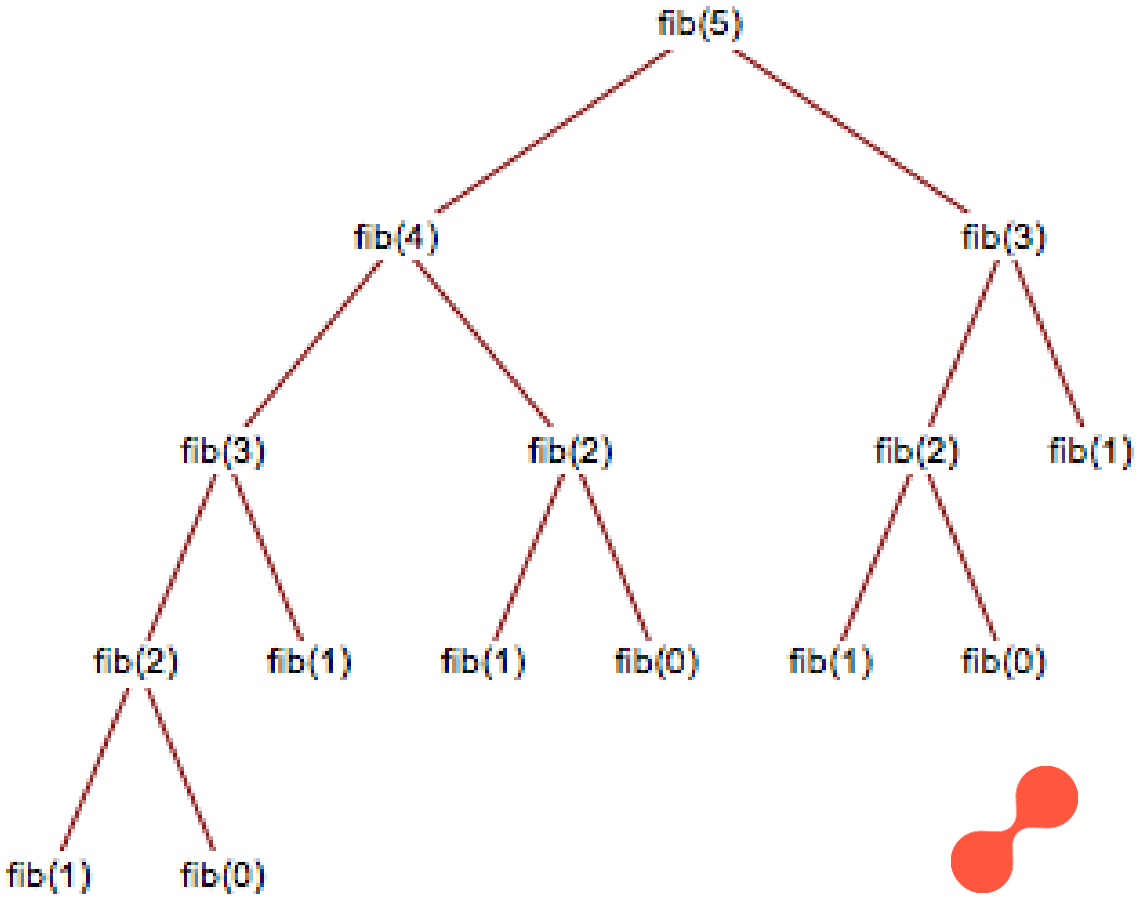
01

¿Que es Programación Dinámica?

Es una técnica para solucionar problemas complejos separándolos en una colección de sub problemas mas sencillos, resolviendo cada uno de ellos solo una vez, guardando los resultados para evitar procesamiento innecesario y repetitivo.

Ejemplo clásico

Secuencia de Fibonacci.



Fibonachi recursivo

- Mucho Trabajo repetitivo.
- Complejidad exponencial

Fibonachi en Programacion DInamica

- Guardamos los resultados
- Complejidad lineal

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

Solución sin PD

Usando recursividad de la función.

- Fib(4) se calcula varias veces.
- Fib(3) tambien.
- La complejidad va creciendo exponencialmente.

```
function fib(n) {
if (n <= 1) {
return n;
}
return fib(n - 1) + fib(n - 2);
}
```

Para `fib(6)` se genera algo así:

```
fib(6)
 ├─ fib(5)
 │   ├─ fib(4)
 │   │   ├─ fib(3)
 │   │   └─ fib(2)
 │   └─ fib(3)
 └─ fib(4)
```

Memoizacion

Concepto importante

Es una técnica que permite la retención de datos recursivos ”cacheándolos”. Usualmente aprovechándonos de el contexto encapsulado para retenerlos.

```
function memoCuadrado() {
const cache = {};

return function(n) {
if (cache[n] !== undefined) {
console.log("Usando cache...");
return cache[n];
}

console.log("Calculando...");
const resultado = n * n;
cache[n] = resultado;
return resultado;
};
}

const cuadrado = memoCuadrado();

cuadrado(4); // Calculando... → 16
cuadrado(4); // Usando cache... → 16
```

Solución con PD

Algoritmo “Bottom-Up”

- Resuelve el problema desde el menor disponible hasta el resultado.
- Usamos iteración en vez de recursión, pasando en orden por cada valor.
- Evitamos el Overhead de llamadas recursivas.

```
//Bottom-Up
const memo = [0, 1];

function fib(n) {
if (n <= 1) return n;

for (let i = 2; i <= n; i++) {
memo[i] = memo[i - 1] + memo[i - 2];
}

return memo[n];
}
```

Programación Dinámica en la practica

¿Como identificar problema de Programación Dinámica?

La PD de forma ideal es un principio que deberías implementar durante etapas tempranas de la concepción de nuestro proyecto, pero en la vida real es común ver que esto falle por multiples motivos, entonces es cuando vemos que la PD surge como una oportunidad para optimizar código que realiza tareas redundantes.

Entonces, **¿Cómo reconocer un caso donde usar PD?.**

- **El problema es divisible.**

  Como problemas parecidos a secuencias de Fibonacci.
- **Notas cálculos que se repiten innecesariamente.**

  Calcular el total del valor de productos en un carrito de compras.
- **Notas que un resultado puede ser re utilizado.**

  Como encontrar la ruta mas rápida entre un lugar y otro.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

“Lo mas importante es tener conciencia de los estados y mantener un enfoque hacia la re utilización y optimización.”